

- ☒ Milan
- ☐ Como
- ☐ GeoInformatics

Exercise 1 [80 points]

Consider the C program below.

```
00  #include <stdio.h>
01  #include <stdlib.h>
02  #include <string.h>
03
04  void usr_in(int sel, char *msg){
05      char usr[4];
06      if (sel){
07          printf("Welcome message: \n");
08          printf(msg);
09          gets(usr); /* Insert username */
10      }
11  }
12
13  void login(int sel, char *msg) {
14      struct {
15          char pass[4];
16          char buf[4];
17          char *error;
18      } s;
19      s.error = "Access Denied!";
20      strcpy(s.pass, "psw\0");
21      usr_in(sel, msg);
22      scanf("%7s", s.buf); /* Insert pass*/
23      if(!strcmp(s.buf, s.pass)){
24          printf("Access Granted!");
25          return 1; /* Access OK */
26      }
27      else {
28          abort(); /* Access KO */
29      }
30  }
31
32  int main(int argc, char** argv) {
33      if (strlen(argv[2]) < 8){
34          login(atoi(argv[1]), argv[2]);
35      }
36      abort();
```

37 }

Assume that:

The C standard library is loaded at a known address during every execution of the program, and that the (exact) address of the function `system()` is `0xfce48da0`.

Environment variables are located in the highest addresses.

8. The program is affected by typical buffer overflow vulnerability. Specify the vulnerable line/s of code. If more than one line is affected, use the following format line1,line2,..,lineN (e.g., 00,01,03). 5 / 5 pt *Valutazione automatica*

09,22



9. Motivate the previous answer.

Punti / 10 pt

The gets function at line 09 has no size check on the input. Therefore, an attacker can overflow the buffer overwrite a portion of memory. The scanf at line 22 has size check, but the limit is greater than the dimension of the buffer. the attacker can overwrite three memory cells past the end of the buffer.

10. The program is affected by typical format string vulnerability. Specify the vulnerable line/s of code. If more than one line is affected, use the following format line1,line2,..,lineN (e.g., 00,01,03). 5 / 5 pt *Valutazione automatica*

08



11. Motivate the previous answer.

Punti / 10 pt

There is no format specification. Therefore an attacker can insert format placeholders to read and write memory locations. Exploit the format string to leak the value of the canary and rewrite it using the buffer overflow vulnerability.

12. Assuming that the program is compiled and run for the usual IA-32 architecture (32-bits), with the usual cdecl calling convention, draw the stack layout just before the execution of line 06, showing: - Direction of growth and high-low addresses; - The name of each allocated variable; - The content of relevant registers (i.e., EBP, ESP); - The functions stack frames (main, login, user_in). Show also the content of the caller frame (just focus on what matters for the exploitation of typical memory corruption vulnerabilities). Use this format: [(H)] [var1 {main ()}] [var2 {main ()}] [sREG (REGY) {main ()}] [sREG1 {fun()}] [var (REGX) {fun()}] [(L)]

Punti / 10 pt

```
[High addr] [*msg {main ()}] [sel {main ()}] [sEIP {main ()}] [sEBP {login ()}]
[*error {login ()}] [buf[0-3] {login ()}] [pass[0-3] {login ()}] [*msg {login ()}]
[sel {login ()}] [sEIP {login ()}] [sEBP (EBP) {usr_in ()}] [usr[0-3] (ESP) {usr_in ()}]
[Low addr]
```

13. Assume that the program is compiled and run with no mitigation against exploitation of memory corruption vulnerabilities (no canary, executable stack, environment with no ASLR active). Write an exploit for the buffer overflow vulnerability identified to bypass the authentication mechanism. Your exploit must execute the following simple shellcode composed only by 12 instructions (12 bytes) that launch the program /bin/secret: 0x31 0xc0 0x6a 0x03 0x5b 0xb0 0x6e 0xcd 0x80 0xfa 0xeb 0xfe. Write the exploit clearly, detail all the steps, input (argv[1] and argv[2]), and assumptions you need for a successful exploitation, and draw the stack layout right before and after the execution of the detected vulnerable line during the program exploitation. Use this format for the answer: [(H)] [var1 | not_relevant] [var2 | not_relevant] [sREG | not_relevant (REGY)] [sREG1 | REGX-2048] [var | 0x.. 0x.. 0x.. 0x.. (REGX)] [(L)]

Punti / 20 pt

```
argv1: any number <> 0 argv2: any string < 8 [High addr | not_relevant]
[*msg | not_relevant] [sel | not_relevant] [sEIP | not_relevant] [sEBP |
shellcode] [*error | shellcode] [buf[0-3] | shellcode] [pass[0-3] | 0x31
0xc0 0x6a 0x03] [*msg | 0x90 0x90 0x90 0x90 (A)] [sel | not_rel] [sEIP |
ESP+16 | address of *msg(login) | addr A] [sEBP (EBP) | (not_relevant)]
[usr[0-3] (ESP) | (not_relevant)] [Low addr] MOTIVATION + ASSUMPTION
```

14. Now assume that the stack is not executable. Is the exploit you just wrote still working without modifications? If not, explain why and propose an alternative solution to launch the program /bin/secret. Write the exploit clearly, detail all the steps and assumptions you need for a successful exploitation, and draw the stack layout right before and after the execution of the detected vulnerable line during the program exploitation. Use this format for the answer: [(H)] [var1 | not_relevant] [var2 | not_relevant] [sREG | not_relevant (REGY)] [sREG1 | REGX-2048] [var | 0x.. 0x.. 0x.. 0x.. (REGX)] [(L)]

Punti / 20 pt

We write in the flag the string /bin/secret, and we overwrite the saved EIP with the address of system(). We then overwrite sel with the ret address of system() and msg with a pointer to ~ESP+24 so that, when jumping into the system() function, this pointer will be at ESP-8 (i.e., where system() expects the parameter). [H_addr] [*msg] |(not_relevant)| [sel=1] | (not_relevant)| [sEIP|ret\0] [SEBP| \sec] [*error| \bin] [buf[0-3]| \\\\] [pas\0| \\\\] [*msg|ESP+24] [sel=1|]*exit() [sEIP|0xfee48da0] [sEBP| <-EBP |(not_relevant)|usr[0-3]| <-ESP |(not_relevant)| [L_addr]

15. Now focus on the format string vulnerability you identified. Is it possible to exploit this vulnerability to overwrite the saved EIP with the address of the executable shellcode (assuming that all addresses are precisely known? If yes, describe precisely how you would exploit this vulnerability and any further assumption you need to make it work. If not, explain why.

Punti / 20 pt

No, the input is limited to 8 bytes, it is not possible to write the full exploit.

16. Now consider that the program is compiled enabling ONLY the exploit mitigation technique known as stack canary (NX and ASLR are not enabled). Assume that the compiler and the runtime correctly implement this technique with a random value changed at every program execution. Is the buffer overflow exploit you wrote at point 2 still working without modifications? Why? If so, please describe precisely why the vulnerability is still exploitable; otherwise, write an alternative exploit that works reliably in this setting (i.e., enabled stack canaries). Please describe precisely how you would modify the above exploit, and any further assumptions (including other vulnerabilities you need to exploit and combine together) you need to make it work.

Exploit the format string to leak the value of the canary and rewrite it using the buffer overflow vulnerability.

Punti / 20 pt

Exercise 2 [100 points]

BookMeUp is a web service that allows students, after having created a personal account, to book office hours with their course instructors. In line with the most recent web development trends, BookMeUp is developed as a thin HTTP Application Programming Interface (API) with a “static” frontend written in HTML+Javascript; as the API is meant to be used programmatically from a JavaScript frontend, all API requests (be it GET or POST ones) return JSON-formatted responses rather than HTML documents. The API is composed of various endpoints (i.e., URLs). You can find below a short description and the implementation (in a Python-like pseudocode) of the endpoints relevant for this exercise.

<https://bookmeup.example.com/api/register> (<https://bookmeup.example.com/api/register>)
(supported method: POST)

Registers a new user (student). Parameters: name, pass.

```
01 def register(request, response):
02     response.set_header('Content-Type: application/json')
03     pass = sha256sum(request.post (http://request.post)['pass'] + config.PASSWD_HASH_SALT)
04     try:
05         db.query("insert into users(`name`, `pass`) " + \
06                 "values ('" + request.post (http://request.post)['name'] + "', '" + pass + "');")
07     except:
08         return json.dumps({'result' : 'fail'})
09     return json.dumps({'result' : 'ok'})
```

<https://bookmeup.example.com/api/login> (<https://bookmeup.example.com/api/login>)

(supported method: POST)

Log in a user. Parameters: name, pass. Sets cookies with the session information (user ID / token).

```
10 def login(request, response):
11     response.set_header('Content-Type: application/json')
12     pass = sha256sum(request.post (http://request.post)['pass'] + config.PASSWD_HASH_SALT)
13     res = db.query("select id, role from users where name = '" + \
14         + request.post (http://request.post)['name'] + "' and pass = '" + pass + "' limit 1;")
15     if res.get_num_rows() != 1:
16         return json.dumps({'status' : 'fail'})
17     uid, role = res.get_first_row()
18     response.set_cookie('user_id', uid)
19     response.set_cookie('signature', sha256sum(uid + config.SECRET_KEY))
20     return json.dumps({'status' : 'ok', 'id' : uid, 'role' : role})
```

<https://bookmeup.example.com/api/slots> (<https://bookmeup.example.com/api/slots>)

(supported method: GET)

Returns a list of the available slots for a given instructor (given its user ID as a GET parameter)

```
21 def slots(request, response):
22     response.set_header('Content-Type: application/json')
23     instructor_id = request.get['instructor_id']
24     res = db.query("select id, timestamp from slots " + \
25         "where user_id = " + instructor_id + " and available = TRUE")
26     slots = []
27     for slot_id, timestamp in res:
28         slots.append({'id' : slot_id, 'time' : timestamp })
29     return json.dumps({'instructor' : instructor_id, 'slots' : slots})
```

<https://bookmeup.example.com/api/book> (<https://bookmeup.example.com/api/book>)

(supported method: POST)

Books an appointment with an instructor. Parameters: instructor ID, appointment slot ID. User must be logged in.

```
30 def book(request, response):
31     response.set_header('Content-Type: application/json')
32     student_id = request.cookies['user_id']
33     if sha256sum(student_id + config.SECRET_KEY) != request.cookies['signature']:
34         return json.dumps({'result' : 'fail'})
35     slot = request.post (http://request.post)['slot']
36     instructor = request.post (http://request.post)['instructor']
37     try:
38         db.query("insert into appointments (`instructor`, `student`, `slot`) " + \
39             "values (" + instructor + ", " + student_id + ", " + slot + ");")
40         db.query("update slots set available = FALSE where slot = " + slot + ";")
41         return json.dumps({'result' : 'ok'})
42     except:
43         return json.dumps({'result' : 'fail'})
```

Assumptions:

The variables request.get and request.post (http://request.post) store the parameters passed through a GET or POST request; request.cookie contains the cookies sent with the request; response.set_header and response.set_cookie allow to add arbitrary headers and cookies to the response. The function json.dumps converts a Python-like dictionary into a string formatted as JSON.

The application uses a relational database for storage, accessible through the function `db.query` via standard SQL queries. Below the database schema:

USERS(id, name, pass, role) *primary key: id (integer autoincrement) - default value for role: "student"*

SLOTS(id, user_id, timestamp, available) *primary key: id (integer autoincrement)*

APPOINTMENTS(slot_id, student_id, instructor_id) *primary key: slot_id and student_id (composite key)*

17. Focus on Cross-site scripting (XSS). Is there a vulnerability belonging to this class in the code?

5 / 5 pt

Valutazione automatica

☐ Yes

☒ No



18. Motivate the previous answer: - If the vulnerability is present, explain -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability. - If the vulnerability is not present, explain why.

Punti / 10 pt

Not applicable, as the API endpoints return a response with MIME type `application/json`, which is not interpreted as HTML (`text/html`) by web browsers (and, as the MIME type is explicitly set in the response headers, it can't be wrongly inferred by the browser from the content). Even if the response had `text/html` MIME type, the application wouldn't contain any XSS: in fact, the only user-controlled value that is inserted in the response is the user ID in the endpoint `/login`, which is an integer (enforced due to the database data type); the user's role and the values output in `/slots` can't be set through the application's functionalities. Likely, they could be set in an administration interface, but we do not have enough information to infer whether any validation or escaping (in this case, unnecessary and wrong!) is being performed there.

19. Focus on Cross site request forgery (CSRF). Is there a vulnerability belonging to this class in the code?

5 / 5 pt

Valutazione automatica

☒ Yes



☐ No

20. Motivate the previous answer: - If the vulnerability is present, explain -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability. - If the vulnerability is not present, explain why.

Punti / 10 pt

(a) Lines: 33-40 (b) Lines 38-40 perform a state-changing action related to the user session, using only ambient credentials (the users' cookies) to authenticate the request. (c) An adversary can exploiting the vulnerability by luring the user to visit an attacker-controlled page (hosted in any domain) which includes a form, pre-populated with the data the attacker wants to use to book an appointment on behalf of the user, and a Javascript snippet to auto-submit the form. The form will be submitted with the user's cookies, if they are logged in. (d) Add an anti-CSRF token as a POST parameter. Requires the API client to perform a request before the POST to retrieve the CSRF token. Alternatively (and better), refactor the application so that the client sends the token via a HTTP header and not via a cookie, so that it isn't an ambient credential. Another option could be the use of same-site cookies.

21. Focus on SQL Injection. Is there a vulnerability belonging to this class in the code?

5 / 5 pt

Valutazione automatica

☒ Yes ✓

☐ No

22. Motivate the previous answer: - If the vulnerability is present, explain -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability. - If the vulnerability is not present, explain why.

Punti / 10 pt

(a) Lines: 5-6, 13-14, 24-25, 38-40 (b) See slides (c) See slides (d) Prepared statements (better) Escaping (worse)

23. Exploiting one of the vulnerabilities detected before, write down an exploit that allows an unauthenticated user to log in as the user "s.zanero".

Punti / 10 pt

At line 14 s.zanero LIMIT 1 ;-- + MOTIVATION/EXPLAIN -----
----- Visit the following URL:
[https://bookmeup.example.com/api/login?](https://bookmeup.example.com/api/login?name=s.zanero'%20limit%201%20;--%20)
[name=s.zanero'%20limit%201%20;--%20](https://bookmeup.example.com/api/login?name=s.zanero'%20limit%201%20;--%20) (%20 is the URL-encoding of a
whitespace. Not required by the exercise)

24. Exploiting one of the vulnerabilities detected before, write down an exploit that allows an arbitrary user to view the user IDs of all the users that have, or had, an appointment with "m.carminati" (without logging in as m.carminati!)

Punti / 10 pt

GET https://bookmeup.example.com/api/slots?instructor_id=<exploit>
<exploit> = " union select student, NOW() from appointments where
instructor_id = (select id from users where name = 'm.carminati'); -- ;"
The returned "slots" will be actually the requested user IDs. Pay attention
to the data types for the union query. ERRATA: due to a typesetting
mistake, the schema of the APPOINTMENTS table was
APPOINTMENTS(slot_id, student_id, instructor) instead of
APPOINTMENTS(slot_id, student_id, instructor_id). If you considered
instructor as a string containing the username of the instructor, the
exploit would be as follows: <exploit> = "000 and false union select
student, 'xxx' from appointments where instructor = 'm.carminati'; -- ;"

25. Assume you are the database administrator, and have no way to modify the source code of the application. Can you mitigate the vulnerabilities you exploited in points 2. and 3.? Which ones? How?

Punti / 10 pt

I could use DB permissions (or avoid the usage of special commands e.g., unions ;-- if allowed by the sql engine). 23: Can't be mitigated, the exploit needs only the permissions that the DB already has. 24: Can be mitigated, The app does not need to SELECT on appointments. We can remove the grant to perform SELECT to the table appointments from the DB user used by the application.

26. Thanks to a marvelous application security team, you fixed all the vulnerabilities found in point 1 right after the discovery. Unfortunately, the BookMeUp security response team receives a message from an instructor, complaining that students are exploiting a vulnerability in the application to book already occupied appointment slots. Please find this vulnerability, describe how it works, how it can be exploited, and what is the simplest way to fix it.

Punti / 10 pt

Logic vuln. /book performs the insertion into appointment without checking if the appointment slot is free. Furthermore the insertion doesn't fail due to any db constraint. Thus we can call /book multiple times. Fix: add a check before the insert (select to retrieve the availability status + if), however this introduces a race condition if the sequence isn't wrapped in a transaction or add a unique constraint on the sb for the column slot_id in the table appointments (or change the primary key to be just the slot_id -- it doesn't make sense that the same slot is booked multiple times!)

27. Now consider the client-side API frontend, written in HTML and Javascript (i.e., client-side code). For simplicity, below you can find a code snippet of the part of the frontend that allows to view the slots of a given user, whose ID is specified in the URL bar (as part of location.hash). For instance, if the current URL is `https://bookmeup.example.com/slots.html#id=10`, the code snippet below will (a) fetch from the API the appointment slots for the instructor with user ID 10, (b) generate an HTML snippet with the data retrieved from the API, (c) insert it into the DOM. `/* location.hash is the part of the URL following the hash (#) e.g., "#id=10" in the above example. */`

```

prof_id =
location.hash.substr(location.hash.search("id=") +
"id=".length) /* fetch() performs a GET request and
handles the result (parsed as JSON) */ url =
'https://bookmeup.example.com/api/slots?instructor_id
= ' + prof_id; response = fetch_and_parse_json(url); /*
result contains the parsed JSON as a Javascript object */
str = "<h1>Available slots for user ID:" + prof_id + "
</h1>"; str += "<table> <tr> <td>ID</td>
<td>Date/time</td> <td> </td> </tr>";
response.slots.forEach((el) => { /* executed for each
element of the array result.slots (ie, it's a for loop) */ str
+= "<tr>" + "<td>" + el.id + "</td>" + "<td>" +
el.timestamp + "</td>" + '<td><a
href="/book.html#id=' + prof_id + '&timestamp=' +
el.timestamp + '">Book now!</td>' + '</tr>'; }) str += "
</table>" document.write(str); Clearly, the code
introduces an additional vulnerability. Identify the class
of the vulnerability, give a brief description of it, and
write an exploit that results in executing the following
code as soon as it visit a certain URL:
alert(document.cookie) Detail all the steps and
assumptions you need for a successful exploitation, and
write the URL that triggers the exploit when visited.

```

Dom based XSS `https://bookmeup.example.com/slots.html#id=`
`<script>alert(document.cookie)</script>`

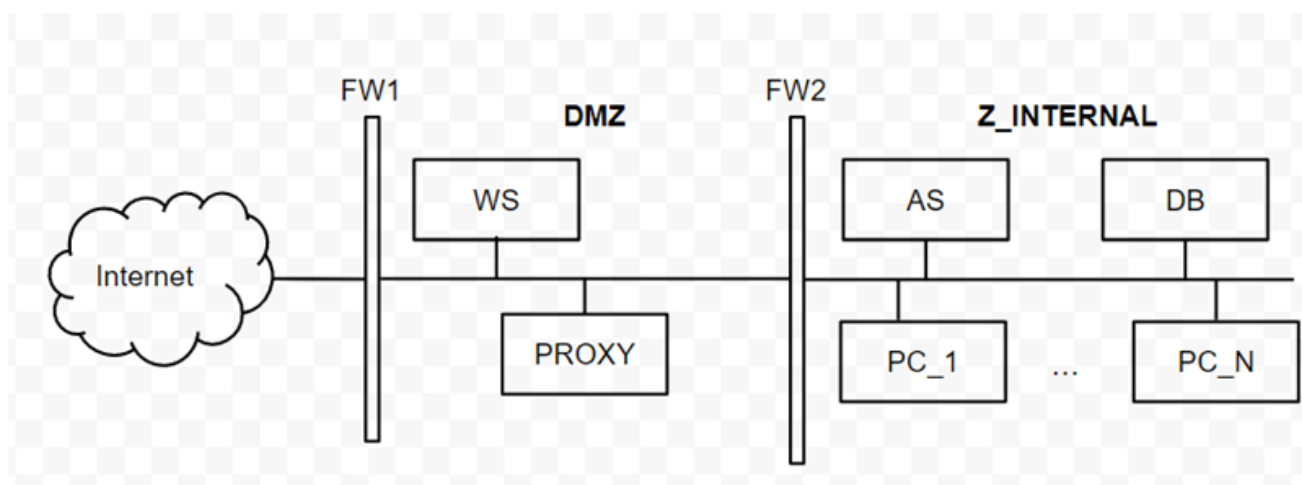
Exercise 3 [60 points]

BeLessSecure, a small IT consultancy company, is rehauling its own local area network. You are in charge of designing the network, taking into account the security requirements.

The company has an external-facing web site, locally hosted on a web server, which should be accessible from the public Internet over the standard HTTP and HTTPS ports (80 and 443, respectively). The web site is, actually, a moderately complex web application, which is supported by an application server that, in turn, uses a MySQL database server as a backend storage. The communication between the web server and the application server happens over TCP, via a standard client\server protocol, where the application server listens on port 8008; the database server listens for connections on TCP port 3308 (i.e., the standard MySQL port).

Besides the servers, the company network contains the employees' desktop computers. Employees can access the application server directly through a Windows software already installed by corporate IT on each computer, using the same protocol that the web server uses. However, employees cannot browse the public web directly: they must use a web HTTP(S) proxy that monitors and filters the web navigation according to corporate policies.

Below the network layout.



28. Write the firewall rules, assuming firewalls to be stateful packet filters. Use the following format: Firewall | Src IP:Src PORT | Direction of the 1st packet | Dst IP:Dst PORT | Policy | Description \\ (example) FW1 | 10.0.0.1:ANY | zone 1 -> zone 2 | 192.168.0.2:443 | DENY | the X server in zone 1 cannot contact the Y server \\

Punti / 20 pt

```
FW1 | ANY:ANY | ANY | ANY:ANY | DENY | DENY ALL \\ FW2 | ANY:ANY | ANY | ANY:ANY | DENY | DENY ALL \\ FW1 | ANY:ANY | WWW->DMZ | WS:80/443 | Allow | WS must be reachable from www \\ FW2 | WS:ANY | DMZ->Z1 | AS:8008 | Allow | AS must be communicate with WS \\ FW2 | PC_IP:ANY | z1->DMZ | PROXY:80/443 | Allow | employee must communicate with proxy \\ FW1 | PROXY:ANY | DMZ->WWW | ANY:80/443 | Allow | proxy must communicate with WWW \\
```

29. BeLessSecure is observing anomalous traffic on its network: They suspect that a nasty cyberattack is taking place. As an incident responder, your task is to understand what is happening. A preliminary analysis reveals that the anomalous traffic is taking place entirely within the network segment of the employees' computers. Hence, you connect to the administrative interface of the managed Ethernet switch that connects together all the employees' computers. First of all, you observe the live network. You can find below a small excerpt of the network traffic passing through a single physical port of the switch only (port name: FastEthernet/12). You also know that there is just a single device (a desktop computer with MAC address dc:a9:04:43:21:22) connected to the port FastEthernet/12. \$ show live-traffic port=FastEthernet/12 source MAC destination MAC src IP:port > dest IP:port --

```

-----
----- a4:da:ca:66:ea:ae 47:c5:a4:11:94:1d 0.0.0.0:58351
> 192.168.1.1:18233 e5:7d:c6:14:24:cc 47:c5:a4:11:94:1d
0.0.0.0:54181 > 192.168.1.1:46945 99:5b:b3:20:90:34
47:c5:a4:11:94:1d 0.0.0.0:39995 > 192.168.1.1:10025
49:e5:8d:59:f6:ea 47:c5:a4:11:94:1d 0.0.0.0:35653 >
192.168.1.1:37071 e0:62:25:3b:0a:20 47:c5:a4:11:94:1d
0.0.0.0:48334 > 192.168.1.1:51807 f3:b9:32:55:04:be
47:c5:a4:11:94:1d 0.0.0.0:17353 > 192.168.1.1:40661
9e:58:7d:51:8e:c7 47:c5:a4:11:94:1d 0.0.0.0:52059 >
192.168.1.1:12306 c4:ac:1c:40:16:c8 47:c5:a4:11:94:1d
0.0.0.0:63469 > 192.168.1.1:50340 6d:9e:8f:60:7b:e0
47:c5:a4:11:94:1d 0.0.0.0:26917 > 192.168.1.1:50171
6b:78:ef:9f:17:2c 47:c5:a4:11:94:1d 0.0.0.0:45635 >
192.168.1.1:36526 <... output omitted ...>
9a:aa:f3:1b:ad:ab 47:c5:a4:11:94:1d 0.0.0.0:35312 >
192.168.1.1:52734 Besides the network log, you also take
a look at the content of the switch's CAM table, as
follows: $ show cam-table CAM Table (memory usage
256KB / total memory 256KB) -----
----- MAC Address Type Port(s) -----
----- 82:30:97:69:57:60
DYNAMIC FastEthernet/1 47:c5:a4:11:94:1d DYNAMIC
FastEthernet/2 <... output omitted ...> dc:a9:04:43:21:22

```

DYNAMIC FastEthernet/12 a4:da:ca:66:ea:ae DYNAMIC
FastEthernet/12 e5:7d:c6:14:24:cc DYNAMIC
FastEthernet/12 99:5b:b3:20:90:34 DYNAMIC
FastEthernet/12 49:e5:8d:59:f6:ea DYNAMIC
FastEthernet/12 f3:b9:32:55:04:be DYNAMIC
FastEthernet/12 9e:58:7d:51:8e:c7 DYNAMIC
FastEthernet/12 c4:ac:1c:40:16:c8 DYNAMIC
FastEthernet/12 <... output omitted ...> 9a:aa:f3:1b:ad:ab
DYNAMIC FastEthernet/12 dc:a9:04:81:3b:e4 DYNAMIC
FastEthernet/24 Which attack is taking place? Give a
brief explanation of how it works, focusing on what an
attacker needs to do in order to implement it and its
effect.

Mac flooding to fill up the CAM table. Generate a large amount of packet
with spoofed source MAC address Effect: CAM table has limited space,
when full the switch will forward everything to every port, becoming
basically a hub

30. Always considering the previous scenario (see Question 29). What is a possible goal of the attacker, in this specific scenario? Justify your answer.

Punti / 10 pt

Sniffing + MOTIVATE

31. Always considering the previous scenario (see Question 29). Describe if, and how, it is possible to mitigate the attack from the point of view of the manufacturer (and, of course, administrator) of the network equipment (e.g., switches, routers, firewalls).

Punti / 10 pt

Implement PORT security. Limit the number of MAC that can be learnt for each port and/or predefine a set of MAC addresses for each port.

Exercise 4 [70 points]

32. A new ransomware variant just broke out, causing a world-wide infection and a huge amount of damages. First of all, you are able to retrieve and disassemble a sample. address binary disassembled instruction

```

8049177: 53 push ebx
8049178: 83 ec 08 sub esp,0x8
804917b: b8 07 00 00 00 mov eax,0x7
8049180: b9 00 00 00 00 mov ecx,0x0
8049185: 0f a2 cpuid
8049187: 83 e3 0f and ebx,0xf
804918a: b8 00 00 00 00 mov eax,0x0
804918f: 83 fb 01 cmp ebx,0x1
8049192: 74 05 je
8049199 8049194: e8 cd ff ff ff call 8049166
<encrypt_disk> 8049199: 83 c4 08 add esp,0x8
804919c: 5b pop ebx
804919d: c3 ret

```

Excerpt from the Intel IA-32 manual: "CPUID returns processor identification and feature information in the EAX, EBX, ECX, and EDX registers. The instruction's output is dependent on the contents of the EAX register upon execution (in some cases, ECX as well)." Additionally, you execute the malware sample in a widespread dynamic analysis environment that, while the program is executing, collects the list of system calls invoked. The log of system calls provided by the sandbox is as follows:

```

execve("./ransomware", ["/ransomware"],
0x7ffd96b406b0 /* 33 vars */) = 0
strace: [ Process PID=372 runs in 32 bit mode. ] [syscalls during program
load, set up of stack/heap/... omitted for the sake of
brevity] exit_group(0) = ? +++ exited with 0 +++

```

Looking at the dynamic traces, it seems that the ransomware is inactive (i.e., it simply terminates once executed). -----

----- What technique is the ransomware implementing? How the technique works?

AntiVM techniques to evade detection. The malware is checking some property of the CPU ...see slides

33. Consider now malware analysis techniques. Briefly discuss which is the best technique to detect the malware behavior implemented by the ransomware considered before.

Static Analysis + MOTIVATION

Before Submitting

34. Do you want your exam to be corrected ?

0 / 0 pt

Select No, if you want to withdraw from the exam.

Valutazione automatica

☒ Yes

☐ No

